Technical Report — Fact Attack: Multilingual NLI over Swiss Voting Booklets

- **Track:** Track 2A — OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Hack Apertus Online (October 1–16, 2026)
- **Team:** Team Fact Attack 2026 — Josip Jukic, Felipe Wüthrich, Sheyla Sampietro, Andrea Petretta
- **Repository:** GitHub · **Experiment log:** docs/experiments.md
- **Demo:** Streamlit app (`make web`: claim check with highlighted evidence pages, benchmark dashboard) and the entailment CLI (`python -m src --input cases.jsonl --output predictions.jsonl`, the Docker image's entrypoint, see 2.5)

1. Summary

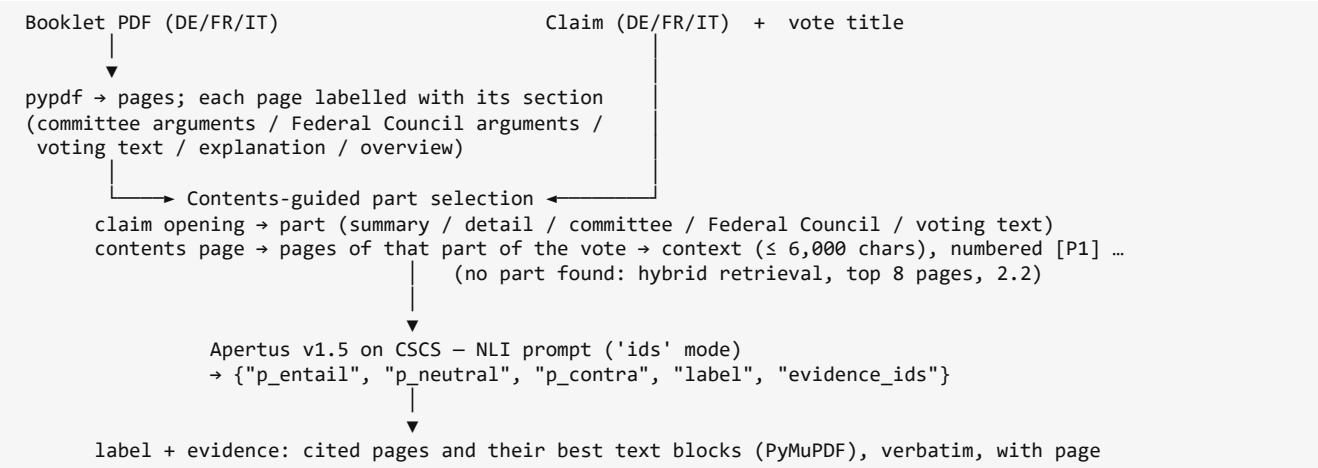
Swiss voters receive an official booklet for every federal vote, and campaigns make claims about it that are hard to check. We built a multilingual claim-verification system on **Apertus v1.5**: given a booklet (German, French or Italian) and a claim (any of the three languages), it classifies the claim as **Entailment (0)**, **Neutral (1)** or **Contradiction (2)** and returns the booklet pages that justify the decision, together with token usage and inference time.

On **booklets the system never saw** (our holdout: 90 claims on the 27 September 2026 vote, labels checked by a second person, used only to confirm the last changes), it reaches **macro-F1 0.956** and **evidence Hit@5 0.933** on the advanced task (booklet PDF) and **0.966** on the beginner task (reference text). On the whole official dataset (v1.1, 1,488 pairs, 20 voting dates in 5 date-grouped folds) the same configuration scores **0.968 / Hit@5 0.975** and **0.984**, at 1,750 / 1,732 input tokens and 59–62 output tokens per case, with processing time outside the LLM calls in the millisecond range. All scores come from the organisers' `evaluate.py` on the official CLI's output.

The official score also weighs evidence, processing time and tokens (relative to the best team), so after the organisers' Q&A we re-tuned for it (section 2.6). The two team members first optimised separately and then merged their lines part by part, each choice decided by paired runs on the same cases: 8 instead of 12 pages (−31 % input tokens, −0.003 F1 on dev), evidence quoted from PyMuPDF pages and text blocks (Hit@5 0.73 → 0.81 on the same predictions), BM25 retrieval with the claim and its translation into the booklet language (macro-F1 0.930 → 0.945 on test), a start-up and parsing path that keeps the scored processing time in milliseconds, beginner-task references cut to the passages that match the claim (−30 % input tokens at equal macro-F1), four compact label rules in the advanced prompt (macro-F1 0.942 → 0.949) and, last, **contents-guided part selection** (2.2): every claim names its source in its opening, and the booklet's contents page locates that part of the vote, which becomes the whole context (macro-F1 0.949 → 0.968, Hit@5 0.881 → 0.975, −55 % input tokens).

The central finding for the challenge's research question: **less, but the right context wins** — the whole booklet (~60k tokens) reaches 0.730 macro-F1, eight retrieved pages (~3,900 tokens) 0.949, and the one part the claim refers to (~1,750 tokens) 0.968.

2. Architecture



2.1 Document processing

`pdf_parser.py` extracts the text of every page with pypdf and detects section headings in all three languages ("Argumente Initiativkomitee", "Arguments du Conseil fédéral et du Parlement", "Il testo in votazione", …). A heading applies to its page and the following pages until the next heading, and table-of-contents entries (heading followed by a page number) are ignored. Section labels matter because booklets deliberately contain opposing viewpoints:

the model has to know whether a page states the committee's or the Federal Council's position. The same pass also reads every page with PyMuPDF (text and text blocks) for the evidence quotes (2.3). Booklets that are not in the image are parsed by a process pool at start-up, one booklet after the other in the order of their cases; the parse of the 60 dataset booklets is baked into the image.

2.2 Context selection: contents-guided part selection

The dataset's claims name their source in their opening ("Laut der Zusammenfassung …", "Le comité estime …", "Se il voto viene accettato …"), and every booklet lists, per vote, the pages of its parts on its contents page ("In Kürze 4 – 5 | Im Detail 14 | Argumente 20 | Abstimmungstext 24", likewise in French and Italian). `part_context.py` combines both:

- **Claim → part:** the earliest cue in the claim's first 80 characters selects summary, committee, Federal Council, voting text or, for the consequences of a yes, the detailed explanation.
- **Contents → pages:** the contents page (also over two pages) gives each part's start page, checked against the heading printed on that page; navigation bars on the summary pages are not mistaken for contents. The arguments are split into committee and Federal Council pages with the parser's speaker labels (2.1).
- **Vote:** the case's vote title is matched by its words to the contents titles and the first lines of each vote's summary and detail pages (contents titles are often abbreviated).
- **Context:** the part's pages in reading order, labelled with the part, at most 6,000 characters; a longer part (voting texts, some detail sections) keeps its best pages by BM25, and only then is a cross-lingual claim translated.
- Coverage: contents read in 60/60 dataset booklets and both holdout booklets; a part found for 1,476 of 1,488 claims. The other claims, and any booklet without a readable contents page, use the hybrid retrieval below.

Fallback: hybrid retrieval (the submission before E12). `retriever.py` scores every page with normalised BM25 against the claim and, for a cross-lingual claim, its translation by Apertus (48 fixed / 25 broken labels over three paired comparisons), plus twice BM25 against the vote title, which is written in the booklet's language and anchors the proposal; it keeps the top 8 pages, clipped to 3,000 characters. If the claim names a side, that side's two best argument pages are always included (speaker boost).

2.3 Apertus as the NLI model

The selected pages are numbered `[P1]…[P8]`. The system prompt defines the three labels, tells the model to judge only against the passages (not world knowledge), to answer Neutral for topics or details the passages do not address, to check claims attributed to an actor only against that actor's text, and to read "recommends rejecting X" as supported when the Federal Council and Parliament recommend "No". For the advanced task, four compact label rules follow (~100 tokens): compare actor, action, object and decisive conditions; Entailment also for paraphrases, summaries and other languages, but never on unstated information; missing support is not a contradiction. Apertus answers with a small JSON object: three confidences, the label, and the ids of the pages that justify it.

- **Evidence by id** guarantees verbatim booklet text with the correct page number and avoids paraphrased quotes.
- **Short answers**: ~55 output tokens instead of ~150 for a JSON answer with reasoning and quotes, which more than halves inference time (p95 5.5 s → 1.8 s on the dev split) at equal or better F1.
- **Evidence items:** up to three cited pages as PyMuPDF reads them, then the two text blocks of the cited pages that best match claim and vote (at most five items). Hit@5 counts an item only if it lies inside the gold passage or contains it; the gold passages follow PyMuPDF's text, so the same blocks built from pypdf or pypdfium2 text reach only 0.80 / 0.73 instead of 0.87 on dev. The PyMuPDF pages are stored with the parse, so no PDF is opened while cases are answered.
- For **Neutral**, the evidence list is empty. If the model predicts 0 or 2 without citing a page, the top-ranked page is used.
- **Decision rules:** in ~2 % of answers the label and the stated confidences disagree. The label is right 3× as often, so it is kept (Rule 0) — except a Neutral label next to a confident other relation, which was right in only 1 of 18 cases and follows the confidences (Rule 0b). Otherwise label and confidences agree and are used as given.

2.4 Beginner task

The reference string (up to 25k characters) is split into passages of ≤ 700 characters at sentence boundaries, numbered and sent with the same prompt, so evidence is a specific passage rather than the whole reference. A reference longer than 6,000 characters is cut to its opening passage plus the passages that best match the claim (BM25 with the claim and, for cross-lingual pairs, its translation), in document order, up to 6,000 characters. On all 1,488 beginner cases this saves 30 % of the input tokens at equal macro-F1 (0.981 → 0.984; 8 fixed / 3 broken claims, against 2 changed labels when the unchanged system runs twice); on the holdout 0.955 → 0.966 at −13 %. The length of a reference is never used as a hint for its label.

2.5 CLI and output format

`python -m src --input cases.jsonl --output predictions.jsonl` (the Docker entrypoint) reads the official case format (JSON or JSONL; a case has `booklet.path + vote` for the advanced task or `reference.text` for the beginner task) as well as rows of the OST dataset on Hugging Face (`claim`, `reference_string`, `booklet_url`, `vote`; also Parquet or CSV), and writes one record per case:

```
{"id": "case-0042", "label": 2, "label_name": "contradiction",
 "evidence": [{"page": 7, "text": "<page 7 as PyMuPDF reads it>"}, {"page": 7, "text": "<best text block>"}, ...],
 "metrics": {"input_tokens": 3983, "output_tokens": 75, "inference_time_ms": 2053}}
```

Evidence is empty for Neutral, and its page is `null` for a reference text.

2.6 Evaluation contract and efficiency

The organisers run the image as `--input /data/cases.jsonl --output /output/predictions.jsonl` with `BASE_URL` (a token-counting proxy) and `API_KEY` injected, `/data` read-only. The score is `0.6 · S_A + 0.4 · S_B`, where S_A = 50 % macro-F1 + 20 % evidence Hit@5 + 15 % processing time + 15 % tokens and S_B = 70 % macro-F1 + 15 % time + 15 % tokens; time and tokens count as best ÷ own over all teams, and processing time is the wall clock minus the time with an Apertus request in flight.

- **Contract:** `BASE_URL` / `API_KEY` take precedence over our older variable names; the parse cache of the 60 dataset booklets is baked into the image read-only and new entries go to `/tmp`; a CI step runs the image exactly this way.
- **Tokens:** the claim's part instead of 8 pages (−55 %, 2.2), after 8 instead of 12 pages (−31 %); beginner references cut to the matching passages (−30 %, 2.4); condensed prompts, narrower windows, a smaller part budget and fewer legal-text pages cost F1 and were rejected (5.3).
- **Processing time:** a standard-library HTTP client (~1 s less start-up), four cases in flight, uncached booklets parsed by a process pool started first: ~2–6 ms per case with cached booklets, ~22 ms on new ones (CLI estimate).
- **Robustness:** a broken case line, an unreadable page, an API error or a stuck parser costs at most its own case, returned as a valid Neutral record; the exit code stays 0.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-70B-thinking` (the model our CSCS key is authorised for; the thinking phase is not triggered by our prompt, answers start directly with the JSON object).
- **Where it runs:** CSCS inference endpoint (`https://api.inference.cscs.ch/v1`), OpenAI-compatible chat completions, temperature 0, retries with exponential backoff.
- **Role:** Apertus is the only model in the pipeline. It translates cross-lingual claims for ranking (only when a part exceeds its budget or the fallback retrieval runs), performs the NLI classification and selects the evidence pages. Locally: pypdf and PyMuPDF (text, blocks), pypdfium2 (fallback), the contents parser and BM25; no OCR, no embeddings. Apertus 8B v1.5 was used only for a comparison run (5.3); no other model was used for development or evaluation.

4. Data and evaluation protocol

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` — 1,495 human-annotated claim/reference pairs from 60 booklets (2020–2026), balanced labels, ~66 % cross-lingual pairs.
- **Booklets:** downloaded from `bk.admin.ch` via the `booklet_url` field (`python -m src.hf_dataset`).
- **Split by voting date**, so all three language versions of a booklet stay on the same side: **dev** (1,093 pairs, 15 dates) and **test** (402 pairs, 5 dates). Test served several before/after decisions, so it is development data by now, not a holdout.
- **Official scorer:** cases from the starter repo's `prepare_cases.py` (dataset v1.1), scored with its `evaluate.py`. Final measurements run the official CLI on **all 1,488 pairs per task in five folds grouped by voting date** (`scripts/run_decision.py`, which forces the submission settings of `.env.example`), and report per fold, language pair and class, macro-F1 on de-duplicated pairs (38 % of the rows are exact duplicates) and a bootstrap over voting dates. Running the unchanged system twice changes 3 (advanced) / 2 (beginner) of 1,488 labels: the endpoint is nearly deterministic, so paired label changes measure the effect of a change.
- **Holdout on unseen booklets:** the jury evaluates on booklets outside the dataset. We wrote 90 claims (30 per label, 10 per language pair, claim types in the dataset's proportions) on the 27 September 2026 booklets before the system ever ran on them; a second team member checked every label (90/90 agree). It confirmed E2 and

served as a regression check for E8 and E12, never to tune anything. (An earlier set of 36 claims on the same booklets was used for two decisions and later lost.)

5. Results

5.1 Final configuration — official CLI and scorer

Evaluation set	Task	Macro-F1	Hit@5	Input / output tokens	Apertus time (mean / p95)
Holdout, unseen booklets (90 pairs)	Advanced	**0.956**	**0.933**	1,751 / 57	–
	Beginner	**0.966**	–	1,758 / 60	–
All 1,488 pairs (5 date folds)	Advanced	**0.968** (folds 0.954–0.978)	**0.975**	1,750 / 59	1.0 s / 1.2 s
	Beginner	**0.984** (folds 0.968–0.990)	–	1,732 / 62	1.1 s / 1.3 s

Processing time outside the model calls: ~2 ms per case (CLI estimate, cached booklets). Before part selection (8 retrieved pages): 0.949 / Hit@5 0.881 at 3,915 tokens on all pairs, 0.933 / 0.850 on the holdout (6 → 4 wrong labels with part selection). Macro-F1 and Hit@5 by language pair (booklet → claim), all 1,488 pairs per task:

Pair	de→de	de→fr	de→it	fr→de	fr→fr	fr→it	it→de	it→fr	it→it
Advanced F1	0.939	0.933	0.958	0.995	0.948	0.990	0.968	0.974	0.994
Advanced Hit@5	0.970	0.933	0.941	1.000	0.977	0.992	0.981	0.969	1.000
Beginner F1	0.966	0.967	0.972	0.995	0.990	0.995	0.994	0.972	1.000

5.2 Full document vs. selected context (advanced, same 150 test pairs)

Context	Prompt	Macro-F1	Input tokens	Output tokens	Latency p95
Full booklet	JSON	0.730	59,517	202	43.7 s*
Proposal filter, top 5 pages	JSON	0.795	3,408	157	36.2 s*
Hybrid, top 10 pages	ids	0.926	6,023	55	3.8 s
12 pages, clipping, speaker boost, decision rules	ids	0.946	5,713	55	2.8 s
Final: the claim's part (2.2), all 1,488 pairs	**ids**	**0.968**	1,750	59	1.2 s

* measured while three runs shared the endpoint.

The full booklet (up to ~70k tokens) is both the most expensive and the least accurate option; the part the claim names is the cheapest and the most accurate.

5.3 Ablations on the dev split

Change	Effect	Decision
`ids` answer instead of JSON with reasoning + quotes	beginner 0.973 → 0.980, advanced 0.786 → 0.797, p95 5.5 s → 1.8 s	adopted
Hybrid retrieval instead of proposal filter	advanced 0.797 → 0.862	adopted
Viewpoint and recommendation rules in the prompt	advanced 0.862 → 0.870, beginner 0.980 → 0.993	adopted
Decision-Rule 0: keep the label when it disagrees with the confidences	44 fixed / 13 broken over all saved runs; test 0.920 → 0.928	adopted
Clip pages > 3,000 characters to the best-matching window	test 0.925 → 0.928, input tokens 5,725 → 4,696 (p95 12.7k → 7.4k)	adopted
12 instead of 10 pages (after clipping)	all 1,495 pairs 0.912 → 0.931 (51 fixed / 24 broken); +~890 input tokens	adopted
Speaker boost (2.2)	all pairs 0.931 → 0.940 (22 fixed / 8 broken), committee-claim errors 36 → 27, no extra tokens	adopted
Decision-Rule 0b (2.3)	all pairs 0.940 → 0.945 (8 fixed / 1 broken)	adopted

Change	Effect	Decision
Thinking before answering (prefilled reasoning marker)	dev 450: 0.908 → 0.929 (20 fixed / 11 broken); ~1,300 output tokens and ~23 s per claim instead of 59 and ~2 s; a 300-token budget is worse	rejected (efficiency)
Other context and answer variants: 600-character passages, claim translated for the prompt, hiding the other side, verification pass, few-shot, speaker hint, label-only with logprobs, reasoning sentence, confidence thresholds, JSON schema (`response_format`), dropping low-value pages	each broke at least as many answers as it fixed (e.g. passages 0.831 vs. 0.908, JSON schema 0.933 → 0.833)	rejected
Apertus 8B instead of 70B (450 dev pairs)	0.851 vs. 0.908 F1, p95 1.2 s vs. 4.5 s	70B kept (quality first; Apertus latency is not scored)
8 instead of 12 pages (dev 300, official scorer)	F1 0.937 → 0.934, Hit@5 0.70 → 0.76, input tokens 5,370 → 3,721; k=7 / 6: F1 0.924 / 0.904; test F1 0.945 → 0.928	adopted (tokens are scored)
Condensed system prompt (`PROMPT_SHORT`)	dev 300 B 0.980 → 0.850, A 0.937 → 0.781, 6–13 % fewer input tokens	rejected
Evidence: PyMuPDF pages + 2 best text blocks (same predictions)	dev 300 0.792 → 0.872, test 0.727 → 0.813	adopted
Retrieval with claim + translation (`union`, paired real runs)	dev 300 0.934 → 0.944, test 0.930 → 0.945, all 1,488 0.948 → 0.956; ~3 % more tokens	adopted
Paired runs on all 1,488 pairs per task (5 date folds; unchanged system run twice: 3 / 2 labels change):		
E1 / E3 / E11: fewer legal-text pages, 2,000-character windows, shorter lower-ranked pages	A 0.943 → 0.937 / 0.934, 0.949 → 0.943; −3 / −9 / −8 % tokens	rejected
E2: beginner references > 6,000 chars cut to the matching passages	B 0.981 → 0.984, −30 % input tokens; holdout 0.955 → 0.966	adopted
E4 / E5 / E6: evidence ranked with the translation, labels for navigation-bar pages, five text blocks instead of pages	Hit@5 within noise (E4, E5); E6 +0.015 only via the vote title that opens every gold passage (passage body 0.843 → 0.766)	rejected
E7/E8: four compact label rules in the advanced prompt	A 0.942 → 0.949 (22 fixed / 11 broken), Hit@5 0.867 → 0.881, +2.5 % tokens; holdout unchanged (2 / 2)	adopted
E12: contents-guided part selection (2.2)	A 0.949 → 0.968 (56 / 29; date bootstrap +0.005 … +0.035), Hit@5 0.881 → 0.975, −55 % tokens; holdout 0.933 → 0.956	**adopted**
E13: the system prompt's rules in fewer words (−210 tokens)	A 0.968 → 0.952 (16 / 40), B 0.984 → ~0.94; errors turn to contradiction	rejected
E14: part budget 4,000 instead of 6,000 characters	A 0.968 → 0.948 (7 / 37), Hit@5 0.975 → 0.940, −6.5 % tokens	rejected

Full tables: docs/experiments.md and docs/independent_experiments.md. Runs on 150 pairs carry about ±3 F1 points of sampling noise; close decisions were re-run on 450 pairs, the last ones on all pairs. A proxy measured without the model (is the gold page in the context, is the gold text still visible) predicted gains for E1, E3 and E11 that the real runs did not show: the model needs the lower-ranked pages too.

5.4 What we learned about Apertus

- **Selected context beats more context.** With the whole booklet (~60k tokens) Apertus 70B reaches 0.73 macro-F1, with 10 selected pages 0.93, with only the part the claim refers to 0.97. We found no position effect ("lost in the middle"): the full-document runs fail even when the answer is in the first third of the booklet, so the degradation is general distraction.

- **Apertus is over-confident.** Its self-reported confidences are 0 or 1 in 97–99 % of cases, and the probability of the label token is ~1.0 for wrong answers as well as for correct ones (41 wrong vs. 40 correct dev answers). Confidence thresholds cannot filter its errors.
- **Distraction, not missing text, caused most errors.** With 8 retrieved pages the gold page was in the context in 97.9 % of the Entailment/Contradiction cases and in 69 of 85 label errors; removing the other parts and votes (E12) fixed 56 answers. Trimming the part itself (E14) or the prompt's wording (E13) costs accuracy at once.
- **Speaker attribution was a context problem.** Committee claims failed in 13.4 % of cases with retrieved pages and no prompt change helped; with the committee's own part as the context, 3 of 275 fail.
- **Thinking helps, at a high price.** Letting the thinking model reason before its answer fixes half of the remaining dev errors (+0.021 F1 on dev 450), but needs ~22× the output tokens and ~11× the latency. A prompt instruction alone does not make it think first; the reasoning marker has to be prefilled. Selective thinking saves little (it is needed on ~3/4 of claims), and a short hard budget is worse.
- **Apertus 8B** is ~3.5× faster but 6 F1 points weaker than 70B on the same contexts.

6. Limitations

- **Remaining errors** (advanced, 48 of 1,488 pairs): Contradiction → Neutral 15, Contradiction → Entailment 11, Entailment → Contradiction 9, Neutral → Contradiction 8, Entailment → Neutral 5. Claims about the voting text fail most often (22 of 214), as long legal texts exceed the 6,000-character budget.
- **Part selection relies on the claims naming their source** and on the contents page of the booklets (2020–2026, checked on 62 booklets). A claim without such an opening, or a booklet without a readable contents page, falls back to the retrieval of 8 pages (0.949 / Hit@5 0.881). The jury's claims may be phrased differently.
- **Unseen booklets:** the holdout (0.956 advanced, 0.966 beginner) covers one vote with two proposals and 90 claims written by us (± ~0.05). The jury's set may differ in layout and claim style.
- **License:** the evidence quotes need PyMuPDF (AGPL-3.0), so the image as a whole is distributed under the AGPL.
- **Development data:** all 20 dates of the dataset, including the former test split, have been used for decisions; the holdout only confirmed E2 and checked E8 and E12 for regressions.
- **Layout:** tables and charts are read as plain text.
- **Processing time and tokens** are measured by the organisers' proxy; our own numbers are estimates (client-side token counts, in-flight clock). Booklets outside the 60 dataset booklets are parsed at run time (in parallel).
- **Not political advice:** outputs describe the relationship between a claim and the official booklet only.

7. Reproducibility

```
docker build --platform linux/amd64 -t fact-attack .          # pre-parses the 60 dataset booklets
docker run --rm -e BASE_URL -e API_KEY -v "$PWD/cases:/data:ro" -v "$PWD/output:/output" \
  fact-attack --input /data/cases.jsonl --output /output/predictions.jsonl
make test                                                      # unit tests, no API calls
```

Cases come from the starter repo's `prepare_cases.py` (or `scripts/prepare_competition_cases.py` for all pairs with date folds, `scripts/build_holdout.py` for the holdout) and are scored with its `evaluate.py`. `scripts/run_decision.py` runs the submission configuration through the official CLI and keeps predictions, scores and settings in `results/generalization/`; `scripts/report_decision.py` compares two runs. Repeated runs differ by a few pairs.

Settings: `.env.example` lists every setting with its default (a test keeps it equal to `src/config.py`), among them `NLI_TOP_K` (`8`), `PAGE_MAX_CHARS` (`3000`), `PROMPT_MODE` (`ids`), `RETRIEVAL_QUERY_MODE` (`union`), `EVIDENCE_POLICY` (`raw_pages_and_blocks`), `REFERENCE_MAX_CHARS` (`6000`), `A_COMPACT_PROMPT` (`true`), `PART_CONTEXT` (`true`), `PART_CONTEXT_CHARS` (`6000`), `NLI_WORKERS` (`4`).

8. Next steps

1. Claims without a recognisable opening: choose the part from the claim's content (e.g. by asking Apertus once), and long voting texts: select their articles rather than whole pages.
2. The remaining Contradiction → Neutral errors; reasoning at lower cost (thinking: +0.02 F1 at ~11× latency).
3. A larger holdout over several new votes, labelled independently, to narrow the ± 0.05 on unseen booklets.

License

Code: Apache License 2.0 (LICENSE). Documentation, including this report: Creative Commons Attribution 4.0 (CC-BY-4.0), as required by the Hack Apertus terms (section 6). The image includes PyMuPDF (AGPL-3.0), so the image as a whole is distributed under the AGPL-3.0, with this public repository as its complete source.